

Bases de données avec Python

Exploiter une base de données avec Python

Dans ce TP, nous allons reprendre notre base de données d'exemples sur les ALBUMS, mais nous allons utiliser Python pour exécuter et exploiter les requêtes SQL. Notre SGBD sera toujours SQLite : le module python que nous utiliserons se nomme **sqlite3**.

```
Entrée[ ]: import sqlite3
```

Le module étant importé, nous devons réaliser deux actions pour pouvoir commencer à utiliser notre base :

- ouvrir le fichier de base de données
- créer un curseur

Le *curseur* est un objet python offrant des méthodes pour exécuter des requêtes et récupérer le ou les résultats de ces requêtes.

```
Entrée[ ]: bdd = sqlite3.connect("albums3.db")
curseur = bdd.cursor()
```

albums3.db est le nom du fichier contenant la base de données SQLite que nous allons exploiter. Si le fichier n'existe pas, une nouvelle base de données sera créée.

Exécuter des requêtes de sélection

Le principe

Reste ensuite à exécuter notre première requête. Pour cela, nous utiliserons la méthode **execute()** du curseur, la requête étant une chaîne de caractères passée en paramètre.

```
Entrée[ ]: requete = "SELECT * FROM Albums;"
curseur.execute(requete)
```

Pour visualiser le résultat de notre requête, nous utiliserons encore notre curseur. Deux méthodes permettent principalement de le faire :

- **fetchone()** pour récupérer un résultat puis avancer le curseur d'un cran
- **fetchall()** pour récupérer d'un coup tous les résultats.

Regardez les exemples ci-dessous pour mieux comprendre comment fonctionne le curseur : il s'agit littéralement d'un curseur que l'on déplace de résultat en résultat. Vous vous en rendrez compte en exécutant plusieurs fois la cellule ci-dessous.

Entrée[]: `curseur.fetchone()`

Vous constatez que le résultat est un *tuple* dont les éléments correspondent aux attributs sélectionnés : ici c'est *. Il n'est pas facile de se rappeler de l'ordre des attributs. Pour cela vous pouvez faire appel à la propriété :

Entrée[]: `curseur.description`

et pour rendre la réponse plus lisible, une petite liste en compréhension ;). Et voilà les attributs de colonne en clair dans l'ordre où ils apparaissent dans le résultat de la requête !

Entrée[]: `[d[0] for d in curseur.description]`

A présent, le fonctionnement de **fetchall()** ne devrait pas vous étonner : on récupère logiquement un tuple avec tous les résultats.

Astuce Jupyter Lab : Lorsque la cellule de résultat est trop longue, faites un clic droit sur la barre bleue à gauche de la cellule, puis sélectionnez "Enable scrolling for output"

Entrée[]: `curseur.fetchall()`

Si vous avez suivi les instructions précédentes, vous devriez constater qu'il manque des enregistrements. Pourquoi ? Un indice : si vous réexécutez une nouvelle fois la méthode **fetchall()** du curseur, celle-ci ne renverra rien !

Et oui, c'est la notion de curseur qui se déplace au fur à mesure qu'un résultat est donné : les précédents appels de **fetchone()** ont fait avancer le curseur, et de même, **fetchall()** positionne le curseur à la toute fin.

Pour retrouver tous les résultats à nouveau, il faut réexécuter la requête. Evitez donc de mélanger **fetchone()** et **fetchall()** sous peine de ne plus trop savoir où en est le curseur et ce que vous récupérez.

Voici donc le moyen le plus simple de récupérer tous les résultats d'une requête d'un coup.

```
Entrée[ ]: curseur.execute(requete)
           resultats = curseur.fetchall()
           resultats
```

Construire des requêtes à partir de variables python

Nous allons dans l'exemple suivant écrire une fonction **album()**

- qui prend en paramètre un curseur et un classement d'album
- qui renvoie le titre de l'album

Si le classement de l'album ne figure pas dans la table *Albums*, la fonction renverra **None**.

```
Entrée[ ]: def album(curs, nombre):
           requete = "SELECT Album FROM ALBUMS WHERE Classement = ?"
           curs.execute(requete, [nombre])
           r = curs.fetchall()
           if len(r) == 0:
               return None
           elif len(r) == 1:
               return r[0][0]
```

```
Entrée[ ]: album(curseur,1)
```

Explications

Dans cet exemple, nous construisons une requête à partir d'une variable Python. SQLite propose un mécanisme de substitution sécurisé permettant d'injecter une ou plusieurs variables à l'intérieur d'une requête. **C'est ce mécanisme que vous devez utiliser** : ne construisez pas vous même la chaîne de caractère contenant la requête complète, c'est une mauvaise pratique qui vous conduira inévitablement à des problèmes.

Pour utiliser ce mécanisme de substitution, vous devez

- mettre des **?** dans votre requête à l'emplacement de la variable à insérer
- passer en second paramètre la liste des valeurs à substituer dans la requête

C'est simple, fiable et sécurisé, en particulier contre les injections SQL (<https://xkcd.com/327/>) !

Exercice 1

Ecrivez une fonction **artiste()**

- qui prend en paramètre un curseur et un ID d'artiste (identifiant d'artiste)
- qui renvoie le nom de l'artiste correspondant

Si le nom de l'ID d'artiste ne figure pas dans la table *Artistes*, la fonction renverra **None**.

```
Entrée[ ]: def artiste(curs, nombre):  
            # Votre code ici
```

```
Entrée[ ]: # Cellule pour vos essais  
print(artiste(curseur,3))
```

```
Entrée[ ]: assert artiste(curseur,1) == "The Beatles"  
assert artiste(curseur,42) == 'Simon & Garfunkel'  
assert artiste(curseur,1000) == None
```

Exercice 2

Ecrivez une fonction **albums2()**

- qui prend en paramètre un curseur et un nom d'artiste
- qui renvoie la liste des albums de cet artiste

Si le nom de l'artiste ne figure pas dans la table *Artistes*, la fonction renverra **None**.

Remarque : la requête peut être écrite sur plusieurs lignes si elle est placée entre triples guillemets

```
Entrée[ ]: def albums2(curs, nom):  
            # Votre code ici
```

```
Entrée[ ]: # Cellule pour vos essais  
print(albums2(curseur, "Green Day"))
```

```
Entrée[ ]: assert albums2(curseur, "Metallica") == ['Master of Puppets', 'Metallic  
assert albums2(curseur, "Green Day") == ['Dookie', 'American Idiot']  
assert albums2(curseur, "Carlos") == None
```

Exercice 3

Ecrivez une fonction **collection()**

- qui prend 3 paramètres un curseur, un nom et un prenom de collectionneur
- qui renvoie la liste des albums de ce collectionneur

Si le collectionneur ne figure pas dans la table *COLLECTIONNEURS*, la fonction renverra **None**.

Remarque : la requête peut être écrite sur plusieurs lignes si elle est placée entre triples guillemets

```
Entrée[ ]: def collection(curs, nom, prenom):  
            # Votre code ici
```

```
Entrée[ ]: # Cellule pour vos essais  
print(collection(curseur, "GUERIN", "Eric"))
```

```
Entrée[ ]: assert collection(curseur,"THOMAS","Guy") == ['Led Zeppelin II', 'Meet  
assert collection(curseur,"THOMAS","Paul") == None  
assert collection(curseur,"DUPONT","Florence") == ['Axis: Bold as Love
```

Insérer de nouveaux enregistrements

Les requêtes de modification sur la base se font de la même manière que les requêtes de sélection, à une petite subtilité près : après exécution de la requête, il faudra faire appel à la méthode **commit()** de l'objet *bdd* (issu de la connexion) afin que les modifications soient prises en compte dans le fichier de base de données.

Attention : Si vous oubliez l'appel à commit, vos modifications seront perdues lorsque vous quitterez votre programme car elles ne seront pas inscrites dans le fichier de la base de données.

```
Entrée[ ]: requete = """  
INSERT INTO COLLECTIONNEURS  
    (id_collectionneur,nom,prenom)  
VALUES  
    (101,'LE MENTEC', 'Mikael');  
"""  
  
curseur.execute(requete)  
bdd.commit()
```

la propriété **lastrowid** peut être intéressante car elle donne accès à la clé primaire créée automatiquement pour notre nouvel enregistrement. En voici une utilisation :

```
Entrée[ ]: last_id = curseur.lastrowid  
last_id
```

```
Entrée[ ]: requete = "SELECT * FROM COLLECTIONNEURS WHERE id_collectionneur = ?"  
curseur.execute(requete, [last_id])  
curseur.fetchone()
```

A vous de jouer

Exercice 4

Effacez de la table auteur ce dernier enregistrement que nous avons créé.

```
Entrée[ ]: # Votre code ici
```

```
Entrée[ ]: requete = "SELECT COUNT(*) from COLLECTIONNEURS"
curseur.execute(requete)
assert curseur.fetchone()[0] == 100
```

```
Entrée[ ]: curseur.execute("SELECT * FROM COLLECTIONNEURS")
curseur.fetchall()
```

Exercice 5

Ecrivez une fonction **ajout_collectionneur()**

- qui prend 3 paramètres un curseur, un nom et un prenom de collectionneur
- qui ajoute ce collectionneur dans la liste des collectionneurs, son numéro de collectionneur est à déterminer par la fonction `dernier_ID_collectionneur()`.

```
Entrée[ ]: def dernier_ID_collectionneur(curs):
            # Votre code ici

print(dernier_ID_collectionneur(curseur))
```

```
Entrée[ ]: def ajout_collectionneur(curs,nom,prenom):
            # Votre code ici
```

```
Entrée[ ]: # Cellule de test
ajout_collectionneur(curseur,'LE MENTEC','Mikael')

# Affichage des collectionneurs
curseur.execute("SELECT * FROM COLLECTIONNEURS")
curseur.fetchall()
```

Pour Finir

Notre travail sur la BDD exemple est à présent terminé. Afin de fermer le fichier proprement et de s'assurer que les données saisies seront bien inscrites dans le fichier, il faut *impérativement* appeler la méthode **close()** sur l'objet *bdd*:

```
Entrée[ ]: bdd.close()
```

```
Entrée[ ]: curseur
```

A partir de ce moment là, plus aucune opération n'est possible sur la base de données comme le montre la cellule suivante :

```
Entrée[ ]: requete = "SELECT COUNT(*) from ARTISTES"
           curseur.execute(requete)
```

```
Entrée[ ]:
```